

Avaliação de Desempenho - INF01146

Análise Comparativa de Algoritmos de Compressão de Arquivos

Etapa 2

Lucas Soller, Lucas Nogueira

06 de Outubro de 2025

1. Descrição do Objeto Computacional

O objeto de estudo deste trabalho são os algoritmos de compressão de arquivos, especificamente os implementados pelas ferramentas **gzip**, **bzip2**, **xz** e **zstd**. Cada uma dessas ferramentas utiliza um algoritmo diferente para reduzir o tamanho de arquivos, com o objetivo de economizar espaço de armazenamento e otimizar a transferência de dados.

2. Configurações do ambiente

OS: Ubuntu 22.04.5 LTS x86_64

Host: Inspiron 3584

CPU: Intel i3-7020U (4) @ 2.300GHz

GPU: Intel Device 5921

RAM: 4 GB

3. Docker Image and packages

Imagem do Docker: `debian:bookworm-slim@sha256:acd98e6...`

`bzip2=1.0.8-5+b1`

`gzip=1.12-1`

`xz-utils=5.4.1-1`

`zstd=1.5.4+dfsg2-5`

`sysstat=12.6.1-1`

`procps=2:4.0.2-3`

`coreutils=9.1-1`

`linux-perf=6.1.153-1`

`g++=4:12.2.0-3`

4. Geração de Arquivos de Teste

Inicialmente iríamos utilizar `/dev/urandom` para geração de arquivos de forma aleatória mas devido a sua alta entropia algoritmos de compressão tiveram desempenho muito ruim.

Nossa próxima abordagem foi utilizar o algoritmo de Cadeias de Markov, conforme descrito por Kernighan e Pike em *A Prática da Programação* (1999), para a geração de texto.

Na nossa implementação, passamos o tamanho do arquivo final em MB e um arquivo base (*A Tale Of Two Cities*) para geração do texto.

5. Benchmarks (Pseudo-código)

```
sizes = (10, 100, 1000)
```

```
for size in sizes:
```

```
    generate file of size MB
```

```
    for algorithm in (bzip2, gzip, xp, zstd):
```

```
        for attempt in range(3):
```

```
            compress()
```

```
            decompress()
```

```
            save_results()
```

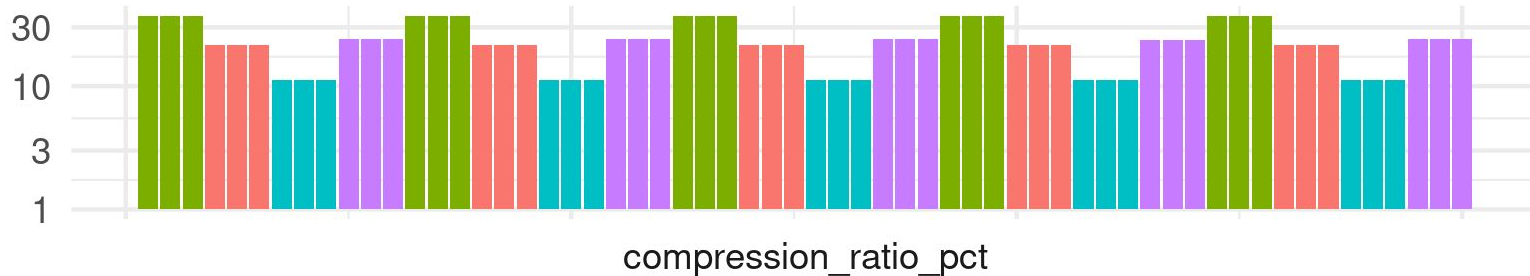
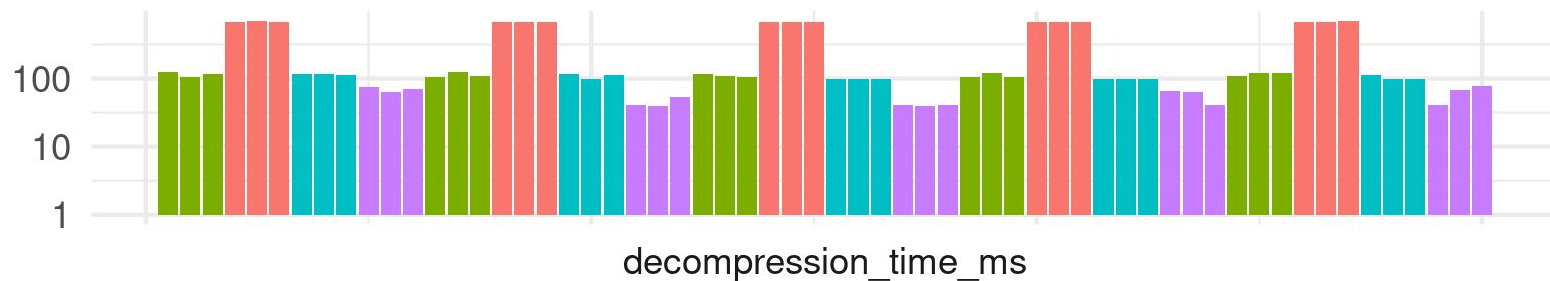
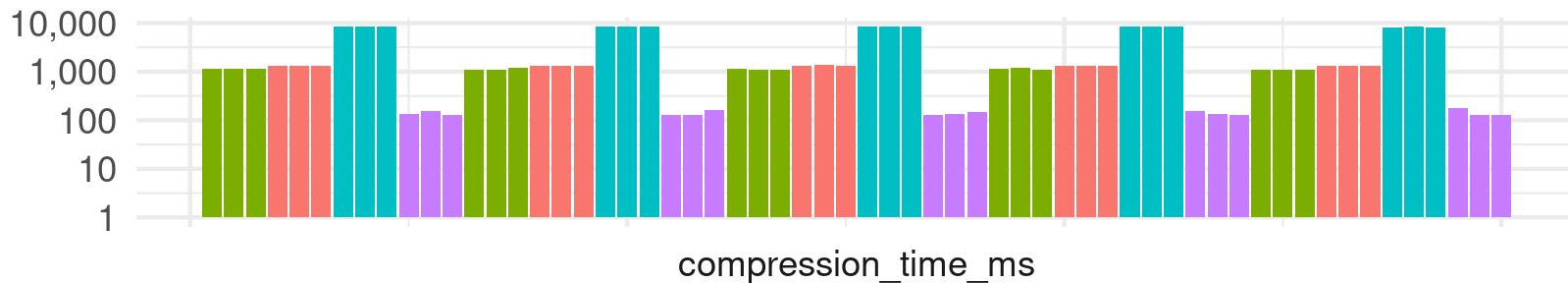
6. Ferramentas

- **Scripts em Bash:** Para automatizar a execução de múltiplas rodadas de testes e a coleta de dados, garantindo a consistência dos experimentos.
- **perf (Linux Performance Events):** Ferramenta avançada para profiling de desempenho que permite coletar dados detalhados sobre a atividade da CPU, como cache misses, instruções executadas e eventos de agendamento do kernel. Usaremos `perf stat` para obter estatísticas resumidas e `perf record` seguido de `perf report` para analisar o comportamento dos algoritmos em nível de código.
- **iostat:** Ferramenta para monitorar a atividade de E/S do sistema (leitura/escrita em disco). Essencial para entender o impacto dos algoritmos nos discos, já que a compressão é uma tarefa intensiva em I/O.
- **vmstat:** Ferramenta para monitorar a memória virtual. Útil para entender o uso de memória (principalmente de swap) durante a compressão e descompressão de arquivos grandes.

7. Métricas Analisadas

- **Tempo de Compressão:** O tempo total (real) que cada ferramenta leva para compactar um arquivo.
- **Tempo de Descompressão:** O tempo que cada ferramenta leva para descompactar o arquivo gerado.
- **Taxa de Compressão:** Calculada pela fórmula (Tamanho final do arquivo comprimido / Tamanho inicial do arquivo) * 100. Esta métrica indica a eficiência de cada algoritmo na redução do tamanho do arquivo.
- **Uso de CPU:** A porcentagem de uso da CPU durante os processos de compressão e descompressão.
- **Uso de Memória:** A quantidade de memória RAM utilizada por cada processo.
- **Atividade de I/O:** A taxa de leitura/escrita em disco durante a execução dos compressores.
- **Eventos de Hardware (com perf):** Dados detalhados de baixo nível, como ciclos de CPU, cache misses (leitura/escrita na memória cache) e número de instruções executadas. Essas métricas fornecerão uma visão profunda da eficiência de cada algoritmo no nível do hardware.

Algorithm Performance 10 MB for Files



algorithm



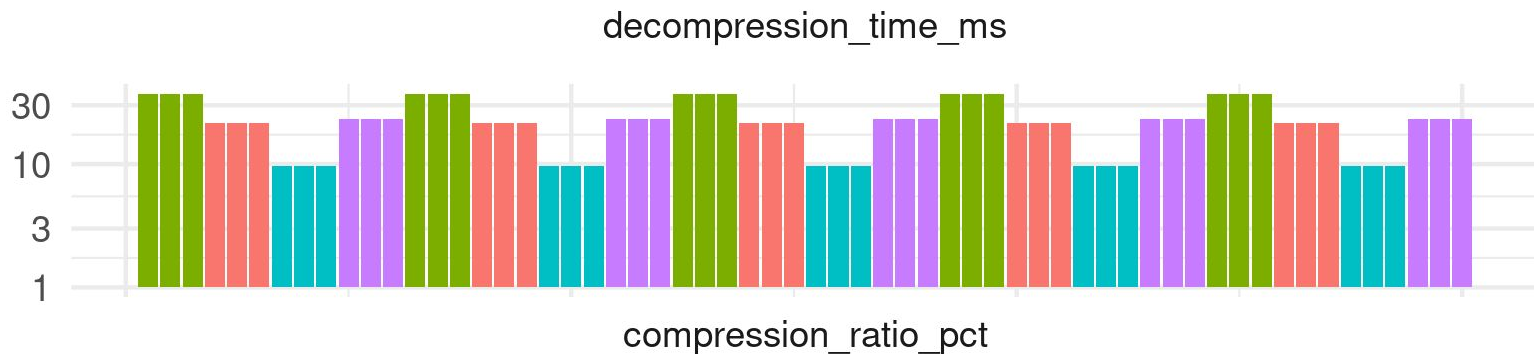
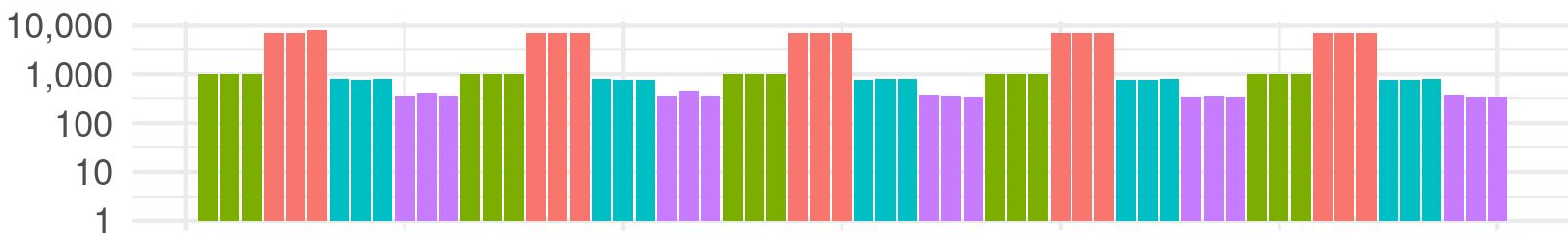
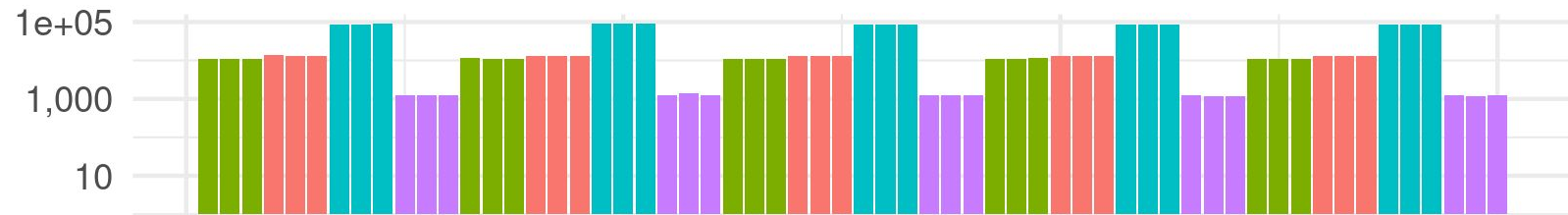
0

20

40

60

Algorithm Performance 100 MB for Files



algorithm



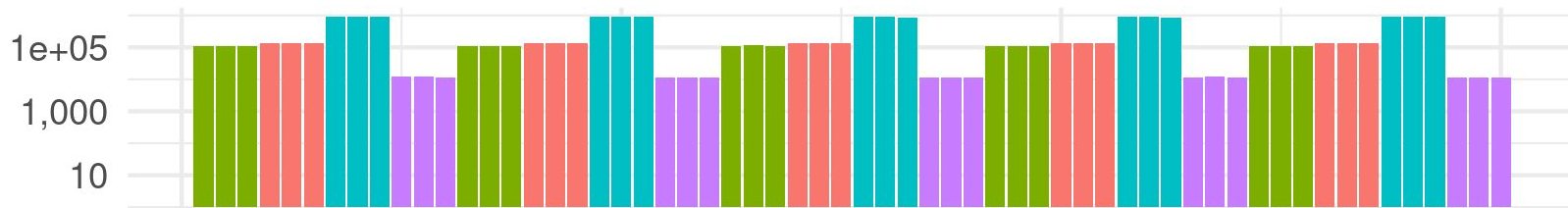
0

20

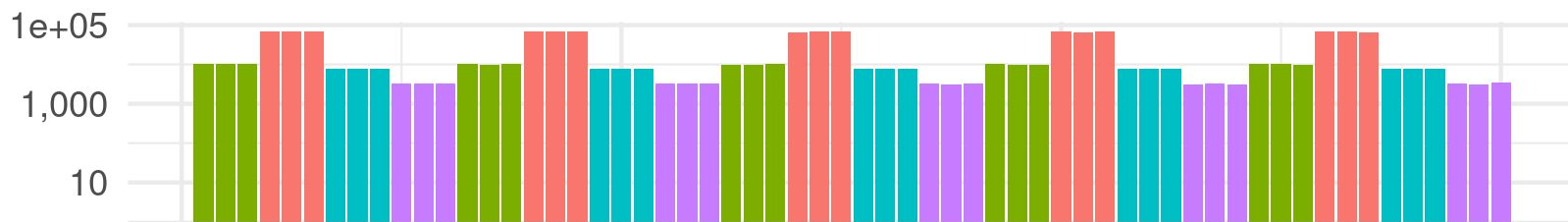
40

60

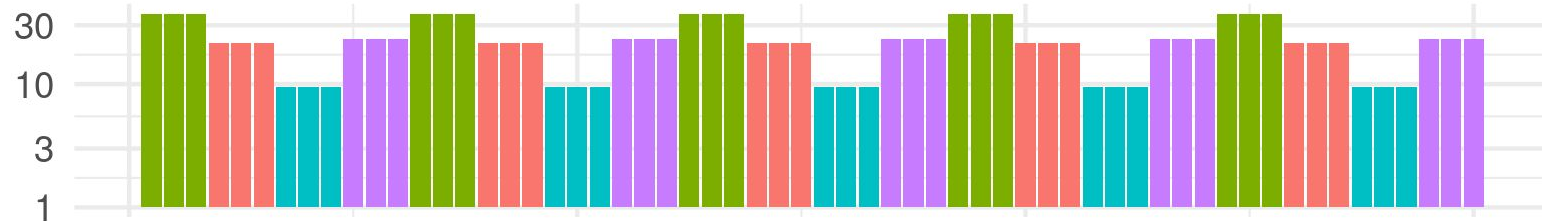
Algorithm Performance 1000 MB for Files



compression_time_ms



decompression_time_ms



compression_ratio_pct

algorithm



0

20

40

60

11. Dificuldades

1. Instrumentação do benchmark
2. Método adequado para geração de arquivos
 - a. Métodos inadequados (**/dev/urandom**)
 - b. Performance
 - c. Dificuldade de implementação

12. Possíveis próximos passos

1. Testar diferentes níveis de compressão de algoritmos
2. Testes com outros tipos de arquivos além de texto (gerados randomicamente)
3. Testar datasets prontos ([squash benchmark](#), Silesia Corpus, Pizza & Chili, etc)

13. Cronograma

Período	Atividade	Entregas e marcos
08/10 a 20/10	Análise Final e Redação do Relatório	Finalizar a análise de dados. Escrever a seção de resultados e discussão, interpretando as métricas e comparando os trade-offs entre os compressores.
21/10 a 27/10	Revisão e Conclusão	Revisar e refinar todo o relatório. Escrever a conclusão do trabalho e adicionar quaisquer detalhes finais.

14. Código Fonte

- <https://github.com/lcdsn/perf-sem25-2>